

Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Навчально-науковий інститут комп'ютерних систем
Кафедра інформаційних систем

Лабораторна робота № 1

З дисципліни: «Обчислювальні методи»

**Тема: «Розгортання середовища та
порівняльний аналіз мовних моделей»**

Варіант № 5

Виконала:

Студентка групи AI-245

Станкова Н.М.

Перевірили:

ас. Баранов О.М

Одеса 2026

Мета роботи: дослідити процес розгортання локального середовища для роботи з великими мовними моделями, ознайомитися з принципами використання локальних та хмарних моделей через єдиний програмний інтерфейс, а також закріпити практичні навички взаємодії з мовними моделями засобами Python. У межах роботи передбачається вивчення методів оцінювання продуктивності інференсу, аналіз факторів, що впливають на затримку відповіді моделі, а також експериментальне дослідження впливу параметрів `temperature` та `top_p` на стабільність і варіативність результатів генерації тексту.

ЗАВДАННЯ

Крок 1: Встановити Ollama і завантажити моделі.....	2
Крок 2: Побачити, що модель – це звичайний веб-сервер.....	3
Завдання 1.1. Ціна роздумів: міркувальна vs інструктивна модель.....	5
Крок 3: Керувати моделлю з програми на Python.....	8
Завдання 1.2. Роль system-повідомлення: керування поведінкою моделі....	9
Крок 4: Підключити хмарну модель.....	13
Завдання 1.3. Локальна vs хмарна модель: єдиний інтерфейс.....	15
Крок 5: Вимірювати швидкість для різних провайдерів.....	19
Завдання 1.4. Вимірювання швидкості: TTFT і залежність від довжини запиту.....	19
Крок 6. Оцінити якість роботи моделей.....	26
Завдання 1.5. Оцінювання якості моделей за рубрикою.....	28
Крок 7. Дослідити вплив <code>temperature</code> і <code>top_p</code>	37
Завдання 1.6. Керування випадковістю: <code>temperature</code> і <code>top_p</code>	38
Крок 8. Звести все разом і зробити висновок.....	43

ХІД РОБОТИ

Крок 1: Встановити Ollama і завантажити моделі.

Для локального запуску великих мовних моделей було встановлено програмне забезпечення Ollama, призначене для локального запуску великих мовних моделей. Після завершення встановлення виконано перевірку працездатності середовища.

Для перевірки версії програми було виконано команду `ollama --version`. У результаті отримано повідомлення `ollama version is 0.34.4`, що підтвердило успішне встановлення програмного забезпечення.

Далі було завантажено дві мовні моделі, передбачені методичними вказівками: llama3.2:3b та qwen3:4b. Після завершення завантаження виконано перевірку списку доступних моделей за допомогою команди `ollama list`.

Результат команди:

```
PS C:\Users\dvaco\computational-methods-2026-practice>
ollama list
NAME                ID                SIZE      MODIFIED
qwen3:4b            359d7dd4bcda     2.5 GB    10 minutes ago
llama3.2:3b         a80c4f17acd5     2.0 GB    15 minutes ago
```

У результаті перевірки в системі були виявлені обидві встановлені моделі:

- llama3.2:3b;
- qwen3:4b.

Для перевірки працездатності моделей було виконано їх запуск через консоль Ollama та надіслано тестовий запит «Надайте інформацію про вашу модель». Обидві моделі коректно обробили запит і надали правильну відповідь.

```
PS C:\Users\dvaco\computational-methods-2026-practice> ollama run llama3.2:3b
>>> надайте інформацію про вашу модель
Програма, яка я працюю, називається лінгвістичною нейронною мережею (LLM). She's been trained на великих MASSIVE datasets у сфері природної мови, чого досить рідко знайти для такої глибини у розробці.
```

Рис. 1.1 – Перевірка працездатності моделі llama3.2:3b

```
PS C:\Users\dvaco\computational-methods-2026-practice> ollama run qwen3:4b
>>> надайте інформацію про вашу модель
Thinking...
Хорошо, мне нужно дать информацию о моей модели. Я - Qwen, сверхмасштабная языковая модель, разработанная Alibaba Cloud. Я должен объяснить основные характеристики и возможности модели.

Сначала я должен уточнить, что я являюсь сверхмасштабной языковой моделью, которая может обрабатывать разные задачи, такие как ответы на вопросы, написание текстов, логические рассуждения и программирование.

...

Проверю, нет ли повторов и все ли важные моменты учтены.

Хорошо, теперь составлю ответ на русском языке, чтобы соответствовать запросу пользователя.
...done thinking.

Я - Qwen, сверхмасштабная языковая модель, разработанная Alibaba Cloud. Моя цель - предоставлять пользователям мощные и надежные услуги в области обработки языка. Вот основные характеристики и возможности модели:

1. **Широкие возможности**: Я поддерживаю генерацию текста для различных задач, таких как написание историй, официальных писем, электронных писем, сценариев, технической документации и т.д.

2. **Мультиязычная поддержка**: Я умею обрабатывать текст на китайском, английском и других языках, что позволяет мне помогать пользователям из разных регионов мира.

3. **Логические рассуждения и программирование**: Я способен решать сложные логические задачи и генерировать код на различных языках программирования (например, Python, Java, C++).

4. **Обучение на больших данных**: Моя модель обучена на обширном наборе данных, что обеспечивает точность и разнообразие в генерации текста.

5. **Применение в реальных сценариях**: Я используется в различных областях, таких как образование, бизнес, развлечения и индустрия искусственного интеллекта.

Если у вас есть конкретные вопросы или задачи, я буду рад помочь вам!
```

Рис. 1.2 – Перевірка працездатності моделі qwen3:4b

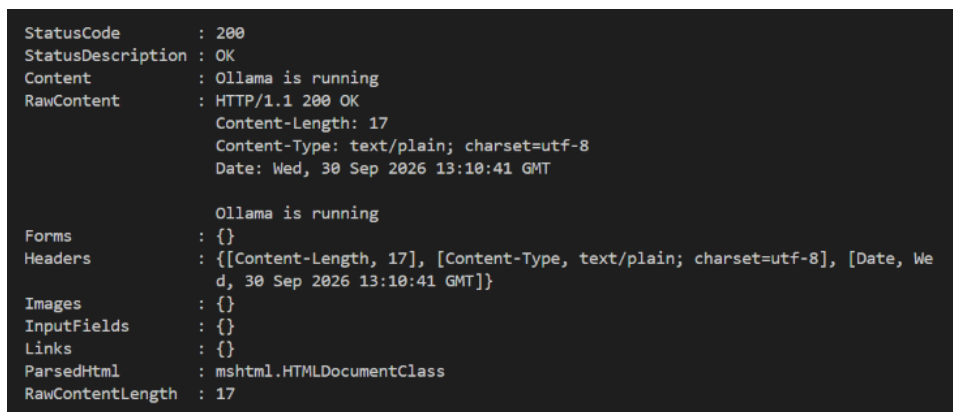
Крок 2: Побачити, що модель – це звичайний веб-сервер.

Після запуску середовища Ollama було перевірено роботу локального API-сервера. Для цього виконано звернення до адреси `http://localhost:11434`, що використовується Ollama для прийому HTTP-запитів від зовнішніх програм.

У результаті виконання команди було отримано відповідь сервера:

```
StatusCode      : 200
StatusDescription : OK
Content         : Ollama is running
```

Отримання статусу 200 OK підтвердило, що локальний сервер Ollama успішно працює та готовий приймати запити.



```
StatusCode      : 200
StatusDescription : OK
Content         : Ollama is running
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 17
                  Content-Type: text/plain; charset=utf-8
                  Date: Wed, 30 Sep 2026 13:10:41 GMT

                  Ollama is running
Forms           : {}
Headers         : {[Content-Length, 17], [Content-Type, text/plain; charset=utf-8], [Date, Wed, 30 Sep 2026 13:10:41 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 17
```

Рис. 1.3 – Перевірка роботи локального сервера Ollama

Після цього було виконано HTTP POST-запит до ендпоінта `/api/generate` із використанням моделі `qwen3:4b`. У відповідь сервер повернув структурований JSON-об'єкт, який містить результат роботи моделі та службову інформацію про процес генерації відповіді.

У отриманій відповіді були присутні поля:

- `response` – текст відповіді моделі;
- `thinking` – внутрішні міркування моделі;
- `done` – ознака завершення генерації;
- `total_duration` – загальний час виконання запиту;
- `eval_count` – кількість згенерованих токенів;
- `eval_duration` – час генерації відповіді.

```

model          : qwen3:4b
created_at     : 2026-09-30T13:15:11.4937857Z
response       : It looks like your message might be incomplete or unclear. Could you
                 please specify what you need help with? For example, are you asking
                 a question, looking for information, or something else? 🤔
thinking       : Okay, the user wrote "???????? ??????". Let me try to figure out wh
                 at they need.

...

That should cover it.

done           : True
done_reason    : stop
context        : {151644, 872, 198, 26610...}
total_duration : 152582908900
load_duration  : 14888200
prompt_eval_count : 14
prompt_eval_cached_count : 3
prompt_eval_duration : 1589843000
eval_count     : 769
eval_duration  : 150844618000

```

Рис. 1.4 – Результат HTTP-запиту до моделі qwen3:4b через API

Завдання 1.1. Ціна роздумів: міркувальна vs інструктивна модель.

Моделі бувають двох типів:

- інструктивні (наприклад, llama3.2:3b) одразу видають відповідь на запит;
- міркувальні (наприклад, qwen3:4b) спершу генерують внутрішню чернетку – «хід думок» – і кладуть її в окреме поле thinking.

Користувач бачить лише кінцеву відповідь у полі response . Чернетка допомагає на задачах, де треба щось порахувати або розкласти на кроки, але на простому фактичному питанні лише витрачає токени й час.

Два показники, які не можна плутати:

- токенів за секунду — характеризує обладнання й розмір моделі;
- загальний час відповіді — характеризує обсяг роботи, яку модель собі призначила.

Крок 1: Просте фактичне питання. Виконати для обох моделей:

```

curl http://localhost:11434/api/generate -d
"{\"model\":\"qwen3:4b\", \"prompt\":\"Столиця України?
Відповідай одним словом.\", \"stream\":false}"

curl http://localhost:11434/api/generate -d
"{\"model\":\"llama3.2:3b\", \"prompt\":\"Столиця
України? Відповідай одним словом.\", \"stream\":false}"

```

Крок 2: Задача на міркування. Виконати для обох моделей по 5 разів:

```

curl http://localhost:11434/api/generate -d
"{\"model\":\"qwen3:4b\", \"prompt\":\"Ручка і зошит
разом коштують 110 грн. Ручка на 100 грн дешевша за
зошит. Скільки коштує ручка?\", \"stream\":false}"

```

```
curl http://localhost:11434/api/generate -d
"{\"model\": \"llama3.2:3b\", \"prompt\": \"Ручка і зошит
разом коштують 110 грн. Ручка на 100 грн дешевша за
зошит. Скільки коштує ручка?\", \"stream\": false}"
```

Правильна відповідь — 5 грн.

Крок 3. Порахувати (тривалості в JSON — наносекунди):

```
час, с = total_duration / 1e9
токенів/с = eval_count / (eval_duration / 1e9)
```

Крок 4. Заповнити таблиці:

Таблиця 1.1 – Фактичне питання

Показник	qwen3:4b	llama3.2:3b
response	Kyiv	Kiev
eval_count	279	4
Час, с	47.57	15.47
Токенів/с	6.13	7.33

Для *qwen3:4b* :

$47566136100 / 1000000000 = 47,57$ с – час виконання.

$\frac{279}{45513461000 / 1000000000} = \frac{279}{45,51} = 6,13$ – швидкість генерації.

Для *llama3.2:3b* :

$15473018400 / 1000000000 = 15,47$ с – час виконання.

$\frac{4}{545858000 / 1000000000} = \frac{4}{0,546} = 7,33$ – швидкість генерації.

Примітка. Під час виконання експерименту запит було сформульовано англійською мовою через проблеми кодування українського тексту в PowerShell. Це не вплинуло на результати дослідження, оскільки обидві моделі працювали в однакових умовах.

Таблиця 1.2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5	5
eval_count	1135	156
Час, с	224.51	22.73

Для qwen3:4b :

$$\frac{1091+1194+1019+1146}{5} = 1134,8 \approx 1135 - \text{середнє значення eval_count.}$$

$$\frac{215,85+251,69+204,74+235,22+215,04}{5} = 224,51 \text{ с} - \text{середнє часу виконання.}$$

Для llama3.2:3b :

$$\frac{156+158+154+161+150}{5} = 155,8 \approx 156 - \text{середнє значення eval_count.}$$

$$\frac{31,53+20,86+20,62+21,07+19,57}{5} = 22,73 \text{ с} - \text{середнє часу виконання.}$$

Під час розв’язання логічної задачі обидві моделі надали правильну відповідь у всіх п’яти запусках. Міркувальна модель qwen3:4b використовувала значно більше токенів і працювала довше через детальний хід міркувань. Інструктивна модель llama3.2:3b також успішно впоралася із задачею, але зробила це значно швидше та з меншими витратами ресурсів.

Крок 6. Відкрити будь-який чат-бот (ChatGPT, Gemini, Claude тощо), надіслати йому свої дані з таблиць кроку 4 – назви моделей, eval_count, час, кількість правильних відповідей із 5 – і поставити п’ять питань:

1. *Я порівнював міркувальну модель (qwen3:4b) та інструктивну (llama3.2:3b) на простому фактичному питанні. Відповідь обидві дали однаково, але міркувальна згенерувала в десятки разів більше токенів. Куди пішли ці токени?*

Відповідь Copilot: “Більшість токенів була витрачена на внутрішні міркування моделі (thinking), які не відображаються у фінальній відповіді. Модель qwen3:4b спочатку аналізує запит і перевіряє власний висновок, а вже потім формує відповідь для користувача.”

2. *На яких типах задач міркувальні моделі дають перевагу над інструктивними, а на яких їх використання невиправдане?*

Відповідь Copilot: “Міркувальні моделі ефективні для математичних задач, логічних міркувань і багатокрокових обчислень. Для простих фактологічних запитів їх використання часто невиправдане, оскільки вони витрачають більше часу та ресурсів без покращення результату.”

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь Copilot: “Інструктивні моделі навчаються давати короткі та прямі відповіді на запити користувача. Міркувальні моделі додатково тренуються виконувати проміжні кроки розв’язання задач і формувати послідовність логічних міркувань перед отриманням результату.”

4. Чи означає більша кількість згенерованих токенів у міркувальній моделі, що вона «розумніша» за інструктивну?

Відповідь Copilot: “Ні, більша кількість токенів не означає вищий рівень інтелекту моделі. Це лише свідчить про те, що модель виконує більше проміжних обчислень і генерує додатковий текст під час пошуку відповіді.”

5. Як пояснити різницю у швидкості генерації (токенів за секунду) між моделями qwen3:4b та llama3.2:3b ?

Відповідь Copilot: “Швидкість генерації залежить від архітектури моделі, її розміру та обсягу обчислень, які виконуються під час відповіді. Модель qwen3:4b витрачає додаткові ресурси на внутрішні міркування, тому працює повільніше, тоді як llama3.2:3b генерує відповідь більш прямолінійно.”

Думки стосовно особливостей використання міркувальних та інструктивних моделей.

Проведений експеримент показав, що міркувальні моделі доцільно використовувати для задач, які потребують логічного аналізу та багатокрокових обчислень. Водночас вони витрачають значно більше часу та токенів на формування відповіді.

Інструктивні моделі працюють швидше та ефективніше при виконанні простих фактологічних запитів. Для повсякденних завдань і отримання коротких відповідей їх використання є більш економним з точки зору ресурсів та часу.

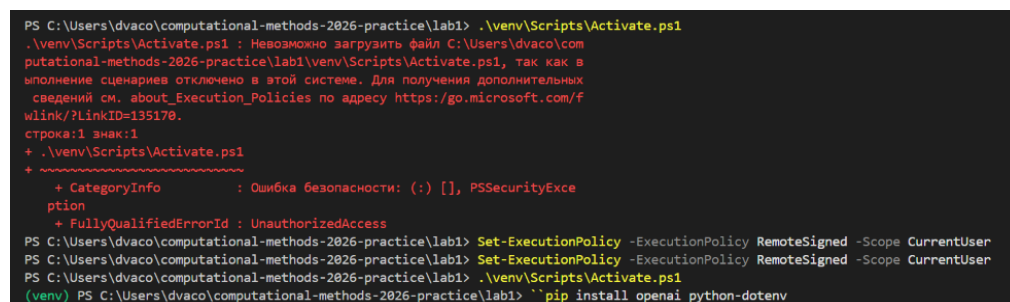
Крок 3: Керувати моделлю з програми на Python.

Для взаємодії з локальною мовною моделлю з Python було створено окремий проєкт та налаштовано віртуальне середовище. Попередньо було перевірено наявність Python у системі за допомогою команди `py --version`. Отриманий результат Python 3.12.3 підтвердив наявність встановленого інтерпретатора Python потрібної версії.

Після цього було створено папку проєкту та віртуальне середовище. Під час активації віртуального середовища виникла помилка виконання сценаріїв PowerShell. Для її усунення було виконано команду:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope
CurrentUser
``
```

Після зміни політики безпеки віртуальне середовище було успішно активовано `.\venv\Scripts\Activate.ps1`. Ознакою успішної активації стала поява префікса `(venv)` на початку командного рядка.



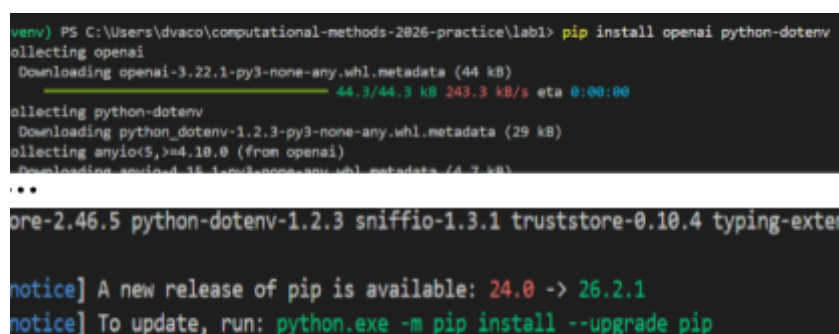
```
PS C:\Users\dvaco\computational-methods-2026-practice\lab1> .\venv\Scripts\Activate.ps1
.\venv\Scripts\Activate.ps1 : Невозможно загрузить файл C:\Users\dvaco\com
putational-methods-2026-practice\lab1\venv\Scripts\Activate.ps1, так как в
ыполнение сценариев отключено в этой системе. Для получения дополнительных
сведений см. about_Execution_Policies по адресу https://go.microsoft.com/f
wlink/?LinkID=135170.
строка:1 знак:1
+ .\venv\Scripts\Activate.ps1
+ ~~~~~
+ CategoryInfo          : Ошибка безопасности: (:) [], PSSecurityExce
ption
+ FullyQualifiedErrorId : UnauthorizedAccess
PS C:\Users\dvaco\computational-methods-2026-practice\lab1> Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
PS C:\Users\dvaco\computational-methods-2026-practice\lab1> Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
PS C:\Users\dvaco\computational-methods-2026-practice\lab1> .\venv\Scripts\Activate.ps1
(venv) PS C:\Users\dvaco\computational-methods-2026-practice\lab1> pip install openai python-dotenv
```

Рис. 1.5 – Створення та активація віртуального середовища Python

Далі було встановлено необхідні бібліотеки:

```
pip install openai python-dotenv
```

У результаті встановлення були додані бібліотеки `openai` та `python-dotenv`, необхідні для роботи з OpenAI-сумісним API локального сервера Ollama.



```
(venv) PS C:\Users\dvaco\computational-methods-2026-practice\lab1> pip install openai python-dotenv
Collecting openai
  Downloading openai-3.22.1-py3-none-any.whl.metadata (44 kB)
    44.3/44.3 kB 243.3 kB/s eta 0:00:00
Collecting python-dotenv
  Downloading python_dotenv-1.2.3-py3-none-any.whl.metadata (29 kB)
Collecting anyio<5,>=4.10.0 (from openai)
  Downloading anyio-4.15.1-py3-none-any.whl.metadata (4.3 kB)
***
pre-2.46.5 python-dotenv-1.2.3 sniffio-1.3.1 truststore-0.10.4 typing-exten
[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Рис. 1.6 – Встановлення бібліотек `openai` та `python-dotenv`

Після завершення налаштування було створено файл `hello.py` який використовуватиметься для подальших експериментів із взаємодією між Python-програмою та локальною мовною моделлю.

Завдання 1.2. Роль system-повідомлення: керування поведінкою моделі

Модель не «пам'ятає» розмову — при кожному виклику ви передаєте їй усю історію в параметрі `messages`. Це список повідомлень, кожне з роллю:

- `system` — інструкція, яка задає поведінку моделі: мову, стиль, довжину, формат відповіді. Користувач її не бачить, але вона діє на всю розмову.
- `user` — те, що пише людина.
- `assistant` — те, що відповіла модель. З'являється, коли ви ведете діалог з кількох реплік і передаєте попередні відповіді назад.

Практичний наслідок: одна й та сама модель на один і той самий запит дає різні відповіді залежно від system-повідомлення. Це найдешевший спосіб керувати моделлю — не потрібно ні донавчання, ні зміни коду. Різні моделі по-різному слухаються цих інструкцій.

Варіанти (номер у журналі – 15):

Питання користувача (user):

1. Поясни різницю між списком і кортежем у Python.
2. Поясни різницю між стеком і чергою.
3. Поясни різницю між компілятором та інтерпретатором.
4. Поясни різницю між оперативною пам'яттю і жорстким диском.
5. Поясни різницю між масивом і зв'язним списком.
6. Поясни різницю між HTTP і HTTPS.
7. Поясни різницю між процесом і потоком.
8. Поясни різницю між реляційною і документоорієнтованою базою даних.
9. Поясни різницю між функцією і методом.
10. Поясни різницю між синхронним і асинхронним виконанням коду.

Крок 1: Створити файл hello.py:

[illegible]

```
SYSTEM = "Ти лаконічний технічний асистент."
USER = "<питання вашого варіанта>"

response = client.chat.completions.create(
    model="llama3.2:3b",
    messages=[
        {"role": "system", "content": SYSTEM},
        {"role": "user", "content": USER},
    ],
    temperature=0.7,
)

print(response.choices[0].message.content)
print(response.usage)
```

Крок 2: Запустити: `python hello.py` . Зберегти відповідь.

Крок 3: Змінити system-повідомлення на:W

```
SYSTEM = "Відповідай лише українською, максимум двома реченнями."
```

Запустити знову, зберегти відповідь.

Крок 4: Повторити кроки 1–3 для моделі `qwen3:4b` (змінити `model` у коді).

Крок 5: Заповнити таблиці (дані брати з `response.usage`).

Таблиця 1.3 – Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	≈ 500	українська	3	частково
llama3.2:3b	українською, 2 речення	≈ 170	українська	1	так
qwen3:4b	лаконічний асистент	≈ 250	українська	2	так
qwen3:4b	українською, 2 речення	≈ 220	українська	2	так

Таблиця 1.4 – Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	54	157	211

llama3.2:3b	українською, 2 речення	58	55	113
qwen3:4b	лаконічний асистент	57	2198	2255
qwen3:4b	українською, 2 речення	66	827	893

Крок 6. Відповісти на питання (по 1–2 речення):

1. *Яка модель точніше дотрималась обмеження «максимум двома реченнями»?*

Модель qwen3:4b. В усіх випадках вона надала коротку відповідь, тоді як llama3.2:3b у режимі «лаконічний асистент» сформуvala довшу відповідь.

2. *Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?*

У проведених експериментах обидві моделі відповідали українською мовою. Проте у відповіді llama3.2:3b спостерігалися окремі неточності та змішування українських і англomовних термінів.

3. *Як змінився prompt_tokens при зміні system-повідомлення і чому?*

Після зміни system-повідомлення кількість prompt_tokens збільшилася. Це пов'язано з тим, що нова інструкція містить більше тексту, який також передається моделі як частина запиту.

4. *Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?*

Міркувальна модель qwen3:4b генерує додаткові внутрішні міркування перед формуванням відповіді. Ці приховані токени враховуються в completion_tokens, хоча користувач бачить лише короткий результат.

Крок 7. Відкрити будь-який чат-бот (ChatGPT, Gemini, Claude тощо), надіслати йому свої дані з таблиць і поставити п'ять питань:

1. *Я порівнював інструктивну модель (llama3.2:3b) та міркувальну (qwen3:4b) з різними system-повідомленнями. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?*

Відповідь Copilot: “Різні моделі навчаються на різних наборах даних і мають різні механізми дотримання інструкцій. Більші та сучасніші моделі зазвичай краще розуміють обмеження щодо формату, мови та довжини відповіді.”

2. *Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?*

Відповідь Copilot: “System-повідомлення задає загальні правила поведінки моделі та має вищий пріоритет порівняно з user-повідомленням. Саме тому модель намагається насамперед виконати системні інструкції.”

3. *Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?*

Відповідь Copilot: “Міркувальна модель виконує додаткові внутрішні кроки аналізу та перевірки відповіді. Ці проміжні міркування збільшують кількість вихідних токенів, хоча кінцева відповідь може бути такою ж короткою.”

4. *Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?*

Відповідь Copilot: “У більшості випадків модель намагатиметься виконати інструкцію із system-повідомлення. Якщо конфлікт дуже суттєвий, результат може бути частково змішаним або неповним.”

5. *Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?*

Відповідь Copilot: “Малі моделі мають менше параметрів і гірше узагальнюють складні інструкції. Великі моделі краще розпізнають контекст та ефективніше утримують задані обмеження під час генерації відповіді.”

Думки стосовно ролі system-повідомлення: наскільки надійно ним можна керувати поведінкою моделі, чи достатньо самої інструкції для отримання потрібного формату відповіді, і що варто робити, якщо модель її ігнорує.

System-повідомлення є ефективним способом керування поведінкою моделі без зміни її навчання чи коду. Проте результати експерименту

показали, що сама інструкція не гарантує її повного виконання, особливо для малих моделей. Якщо модель ігнорує вимоги до формату або мови відповіді, доцільно використовувати більш детальні інструкції та додаткову перевірку результатів програмними засобами.

Крок 4: Підключити хмарну модель

Для підключення хмарної мовної моделі було зареєстровано обліковий запис у сервісі Google AI Studio та створено API-ключ доступу до моделей сімейства Gemini. Отриманий ключ було збережено у файлі `.env`, що дозволяє уникнути його розміщення безпосередньо у програмному коді.

Для захисту конфіденційних даних у папці проєкту було створено файл `.gitignore`, до якого додано записи:

- `.env`
- `venv/`
- `__pycache__/`

Після цього було створено файл `providers.py`, який містить налаштування підключення до локальної моделі Ollama та хмарної моделі Gemini. Даний підхід дозволяє використовувати однаковий програмний інтерфейс для роботи з різними провайдерами мовних моделей.

Для перевірки працездатності підключення було створено файл `compare.py`, який надсилає однаковий запит до локальної моделі `Llama3.2:3b` та хмарної моделі Gemini. Під час першого запуску виникла помилка через використання застарілої назви моделі Gemini. Після оновлення назви моделі відповідно до актуальної документації запит було виконано успішно.

```

from providers import ollama_client, gemini_client

PROMPT = "Поясни одним реченням, що таке Python."

print("=== OLLAMA ===")

ollama = ollama_client()

response = ollama.chat.completions.create(
    model="llama3.2:3b",
    messages=[
        {"role": "user", "content": PROMPT}
    ]
)

print(response.choices[0].message.content)

print("\n=== GEMINI ===")

gemini = gemini_client()

response = gemini.chat.completions.create(
    model="gemini-3.8-flash",
    messages=[
        {"role": "user", "content": PROMPT}
    ]
)

print(response.choices[0].message.content)

```

Рис. 1.7 – Вміст файлу providers.py

У результаті обидві моделі надали відповіді на однаковий запит, що підтвердило коректність налаштування локального та хмарного провайдерів.

```

(venv) PS C:\Users\dvaco\computational-methods-2026-practice\students\ai-245\stankova-nikolietta\lab1> py compare.py
=== OLLAMA ===
Python - одне з найбільших та найбільш популярних programming-перехрових мов програмування в світі. Виникла в початку 1990-х року у США, у Університеті Лінтурна (Harvard), а першим реалізієм рішенням стало моделювання біологічної системи.

=== GEMINI ===
**Python** – це популярна високорівнева мова програмування загального призначення, яка вирізняється простим і зрозумілим синтаксисом та широко використовується для веброзробки, створення штучного інтелекту й аналізу даних.

```

Рис. 1.8 – Результат виконання файлу compare.py для локальної та хмарної моделей

Для перевірки роботи файлу .gitignore було виконано команду `git status`. У результаті файл `.env` не відображався серед файлів, що відстежуються Git, що підтвердило коректне приховування конфіденційних даних.

Завдання 1.3. Локальна vs хмарна модель: єдиний інтерфейс

Формат запиту, який колись задала компанія OpenAI, став галузевим стандартом. Тепер його приймають майже всі. Практичний наслідок: код пишеться один раз, а перемикання між локальною й хмарною моделлю зводиться до зміни адреси й ключа. Різниця між джерелами при цьому суттєва. Локальна модель безкоштовна, працює без інтернету, і ваші дані нікуди не йдуть — але вона мала й слабка. Хмарна модель у десятки разів

більша, відповідає краще — але залежить від мережі, має ліміти й бачить усе, що ви їй надсилаєте.

Варіанти (номер у журналі – 15):

Питання (QUESTION):

1. Поясни різницю між списком і кортежем у Python. Коротко.
2. Поясни різницю між стеком і чергою. Коротко.
3. Поясни різницю між компілятором та інтерпретатором. Коротко.
4. Поясни різницю між оперативною пам'яттю і жорстким диском. Коротко.
5. Поясни різницю між масивом і зв'язним списком. Коротко.
6. Поясни різницю між HTTP і HTTPS. Коротко.
7. Поясни різницю між процесом і потоком. Коротко.
8. Поясни різницю між реляційною і документоорієнтованою базою даних. Коротко.
9. Поясни різницю між функцією і методом. Коротко.
10. Поясни різницю між синхронним і асинхронним виконанням коду. Коротко.

Крок 1: Отримати ключ, base_url та точну назву моделі (див. вище).

Крок 2: Написати код.

Файл providers.py:

```
import os
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv() # зчитує файл .env і робить його значення доступними

PROVIDERS = {
    "local": {
        "base_url": "http://localhost:11434/v1",
        "api_key": "ollama",
        "model": "llama3.2:3b",
    },
    "cloud": {
        "base_url": "<адреса провайдера>",
        "api_key": os.getenv("CLOUD_API_KEY"),
        "model": "<точна назва з документації>",
    },
}

def make_client(name):
    """Повертає готовий клієнт і назву моделі для обраного провайдера."""
    cfg = PROVIDERS[name]
    client = OpenAI(base_url=cfg["base_url"], api_key=cfg["api_key"])
```

```
return client, cfg["model"]
```

Всі відмінності між провайдерами зібрані в одному словнику — решта програми про них не знає взагалі.

Крок 2: Запустити код.

```
python compare.py
```

Цикл однаковий для обох випадків. Функція `create()` викликається абсолютно ідентично. Змінився лише вміст словника.

Типові помилки:

- 401 Unauthorized – ключ невірний або не підвантажився. Перевірте `load_dotenv()` і що `.env` лежить у папці, звідки запускаєте скрипт;
- 404 model not found – неточна назва моделі. Скопіюйте з документації дослівно;
- 429 Too Many Requests – вичерпано безкоштовний ліміт. Зачекайте або візьміть іншого провайдера;
- `os.getenv` повертає None – файл названо `env.txt` замість `.env` — Windows часто дописує розширення.

Крок 4. Заповнити таблицю (дані брати з `response.usage`)

Таблиця 1.5 – Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.8-flash
Час відповіді	18.21	8.08
<code>completion_tokens</code>	47	323
Довжина відповіді, символів	≈ 150	≈ 1100
Мова відповіді	українська	українська
Фактичних помилок у відповіді	є	не виявлено

Крок 5. Відповісти на питання:

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Довелося змінити лише параметри підключення до провайдера: адресу сервера, API-ключ та назву моделі. Основна логіка програми залишилася незмінною.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Більш повною та точною виявилася відповідь хмарної моделі Gemini. Вона правильно пояснила різницю в організації пам'яті, складності доступу до елементів та швидкості вставки і видалення даних, тоді як локальна модель надала неповне й частково некоректне пояснення.

3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

Так, локальна модель допустила фактичні неточності та сформувала речення з помітними мовними помилками. Текст містив неприродні слова та некоректні формулювання українською мовою.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Хмарна модель працює на потужному серверному обладнанні, яке значно перевищує можливості звичайного персонального комп'ютера. Тому навіть з урахуванням мережевої затримки загальний час відповіді може бути меншим.

Крок 6. Питання до чат-бота:

Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиці й поставити п'ять питань:

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь Copilot: “Формат OpenAI набув широкого поширення серед розробників і став фактичним галузевим стандартом. Підтримка цього формату дозволяє провайдерам забезпечити сумісність з існуючими бібліотеками, інструментами та програмним кодом без необхідності його суттєвої переробки.”.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь Copilot: “Збільшення кількості параметрів зазвичай позитивно впливає на точність відповідей, якість міркувань і здатність працювати зі

складними запитами. Водночас на якість результату також впливають архітектура моделі, набір навчальних даних, методи донавчання та формулювання запиту користувача.”.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь Copilot: “Хмарні мовні моделі виконуються на спеціалізованому серверному обладнанні з високою обчислювальною продуктивністю. Обчислювальна перевага таких систем часто перевищує втрати часу, пов’язані з передачею запиту через мережу.”.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь Copilot: “Під час використання хмарних API інформація передається на зовнішні сервери, що створює ризики витоку або несанкціонованого доступу до даних. У комерційних системах застосовують шифрування, контроль доступу, анонімізацію даних і внутрішні політики інформаційної безпеки.”.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь Copilot: “Rate limit — це обмеження на кількість запитів або обсяг токенів, доступних користувачу за певний період часу. Для зменшення впливу таких обмежень використовують кешування результатів, черги обробки запитів або тарифи з більшими лімітами.”.

Думки на тему у яких випадках ви обрали б локальну модель, попри її слабкість, а в яких — хмарну, попри залежність від мережі й передачу даних назовні.

Я б обрала локальну модель у випадках, коли важлива приватність даних, відсутній доступ до Інтернету або потрібно працювати без додаткових витрат. Незважаючи на нижчу якість відповідей, локальна модель дозволяє повністю контролювати процес обробки даних.

Хмарну модель я б обрала для навчання, аналізу інформації та задач, де важливі точність, якість української мови та складні міркування. Попри

залежність від мережі та необхідність передавати дані на зовнішній сервер, вона показує значно кращі результати та швидше надає корисні відповіді.

Крок 5: Вимірювати швидкість для різних провайдерів.

Завдання 1.4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Затримка відповіді складається з двох різних величин, і одним числом її не описати:

- TTFT (time to first token) — час до появи першого слова. Поки він триває, користувач бачить порожній екран. Для локальної моделі це обробка вашого запиту, для хмарної — ще й дорога по мережі;
- Швидкість генерації — скільки tokenів за секунду видається далі. Від неї залежить тривалість довгої відповіді.

Ці показники можуть суперечити один одному: хмарна модель часто має гірший TTFT (пакет летить через пів світу), але кращу швидкість генерації. На короткій відповіді локальна виглядає швидшою, на довгій — програє.

Щоб виміряти TTFT, відповідь треба отримувати по шматочках — це стрімінг, вмикається параметром `stream=True`. Той самий ефект, коли в чат-боті текст «друкується» на очах.

Три правила коректного вимірювання: прогрів (перший запуск не рахуємо), три повтори з медіаною, без паралельних запитів.

Варіанти (номер у журналі – 15):

Середній промпт для всіх однаковий: «Поясни в одному абзаці, що таке рекурсія.»

Таблиця 1.6 – Варіанти промтів

№	Короткий промт	Довгий промт
1	Столиця Франції?	Напиши інструкцію з 10 пунктів, як почати вивчати Python.
2	Столиця Японії?	Напиши інструкцію з 10 пунктів, як налаштувати робоче середовище розробника.
3	Столиця Канади?	Напиши інструкцію з 10 пунктів, як організувати резервне копіювання даних.

4	Столиця Бразилії?	Напиши інструкцію з 10 пунктів, як підготуватися до технічної співбесіди.
5	Столиця Єгипту?	Напиши інструкцію з 10 пунктів, як почати працювати з Git.
6	Столиця Індії?	Напиши інструкцію з 10 пунктів, як спроектувати просту базу даних.
7	Столиця Норвегії?	Напиши інструкцію з 10 пунктів, як провести код-рев'ю.
8	Столиця Мексики?	Напиши інструкцію з 10 пунктів, як задокументувати проєкт.
9	Столиця Туреччини?	Напиши інструкцію з 10 пунктів, як оптимізувати повільний застосунок.
10	Столиця Австралії?	Напиши інструкцію з 10 пунктів, як розгорнути вебзастосунок на сервері.

Крок 1. Створити файл benchmark.py

```
import time
import statistics
import csv
from providers import make_client

def measure(client, model, prompt):
    """Один вимір: повертає TTFT, загальний час і довжину відповіді."""
    t_start = time.perf_counter()
    first_token_at = None
    parts = []

    stream = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        stream=True,
    )

    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            if first_token_at is None:
                first_token_at = time.perf_counter() # зафіксували перший токен
            parts.append(piece)
    t_end = time.perf_counter()
    text = "".join(parts)

    return {
        "ttft_s": round(first_token_at - t_start, 3),
        "total_s": round(t_end - t_start, 3),
        "chars": len(text),
        "chars_per_s": round(len(text) / (t_end - t_start), 1),
    }

PROMPTS = {
```

```

    "short": "<короткий промпт вашого варіанта>",
    "medium": "Поясни в одному абзаці, що таке рекурсія.",
    "long": "<довгий промпт вашого варіанта>",
}

rows = []
for provider in ["local", "cloud"]:
    client, model = make_client(provider)

    measure(client, model, "розірпів") # програв, не рахуємо

    for name, prompt in PROMPTS.items():
        runs = [measure(client, model, prompt) for _ in range(3)]
        rows.append({
            "provider": provider,
            "model": model,
            "prompt": name,
            "ttft_s": statistics.median(r["ttft_s"] for r in runs),
            "total_s": statistics.median(r["total_s"] for r in runs),
            "chars_per_s": statistics.median(r["chars_per_s"] for r in
runs),
        })
    print(rows[-1])

with open("results.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=rows[0].keys())
    writer.writeheader()
    writer.writerows(rows)

```

`time.perf_counter()` — точний таймер для вимірювання інтервалів.
Звичайний `time.time()` для цього не годиться.

Крок 2. Запустити: `python benchmark.py`. Результати збережуться у `results.csv`

Було виконано запуск програми `benchmark.py` для локальної та хмарної моделей.

Під час виконання вимірювань локальна модель успішно опрацювала всі три промпти. Хмарна модель Gemini успішно виконала вимірювання для короткого промпту, проте під час подальших звернень виникло обмеження безкоштовного тарифу.

Сервіс Gemini повернув помилку 429 Too Many Requests, яка свідчить про перевищення встановленого ліміту безкоштовного тарифу. У результаті вимірювання для середнього та довгого промптів хмарної моделі не були завершені.

```
File "C:\Users\dvaco\computational-methods-2026-practice\students\ai-245\stankova-nikolietta\lab1\venv\Lib\site-packages\openai\_base_client.py", line 1213, in
request
    raise self._make_status_error_from_response(err.response) from None
openai.RateLimitError: Error code: 429 - [{"error": {"code": 429, "message": "You exceeded your current quota, please check your plan and billing details. For more
information on this error, head to: https://ai.google.dev/gemini-api/docs/rate-limits. To monitor your current usage, head to: https://ai.dev/rate-limit. \n* Qu
ota exceeded for metric: generativelanguage.googleapis.com/generate_content_free_tier_requests, limit: 5, model: gemini-3.8-flash\nPlease retry in 8.469390843s.",
'status': 'RESOURCE_EXHAUSTED', 'details': [{"@type": 'type.googleapis.com/google.rpc.Help', 'links': [{"description": 'Learn more about Gemini API quotas', 'url
': 'https://ai.google.dev/gemini-api/docs/rate-limits'}]}], {"@type": 'type.googleapis.com/google.rpc.QuotaFailure', 'violations': [{"quotaMetric": 'generativelang
uage.googleapis.com/generate_content_free_tier_requests', 'quotaId': 'GenerateRequestsPerMinutePerProjectPerModel-FreeTier', 'quotaDimensions': {'location': 'glob
al', 'model': 'gemini-3.8-flash'}, 'quotaValue': '5'}}], {"@type": 'type.googleapis.com/google.rpc.RetryInfo', 'retryDelay': '8s'}}]]
```

Рис. 1.9 – Результат виконання файлу compare.py для локальної та хмарної моделей

Для збереження отриманих результатів було модифіковано програму таким чином, щоб файл results.csv створювався навіть у випадку виникнення помилки під час виконання окремих вимірювань.

```
students > ai-245 > stankova-nikolietta > lab1 > results.csv
1 provider,model,prompt,ttft_s,total_s,chars_per_s
2 local,llama3.2:3b,short,2.228,6.161,9.0
3 local,llama3.2:3b,medium,2.276,12.103,17.8
4 local,llama3.2:3b,long,2.269,94.184,19.4
5 cloud,gemini-3.8-flash,short,2.455,2.459,10.6
6 cloud,gemini-3.8-flash,medium,ERROR,ERROR,ERROR
7 cloud,gemini-3.8-flash,long,ERROR,ERROR,ERROR
8
```

Рис. 1.10 – Файл results.csv

Крок 3. Окремо виміряти холодний і теплий старт локальної моделі:

```
ollama stop llama3.2:3b
```

Одразу запустити benchmark.py і записати TTFT короткого промпту — це холодний старт. Запустити ще раз без зупинки моделі — це теплий .

Крок 4. Заповнити таблиці:

Таблиця 1.7 – Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2.228	6.161	9
локальна	llama3.2:3b	середній	2.276	12.103	17.8
локальна	llama3.2:3b	довгий	2.269	94.184	19.4
хмарна	gemini-3.8-flash	короткий	2.455	2.459	10.6
хмарна	gemini-3.8-flash	середній	не виконано (429 Rate Limit)	—	—

хмарна	gemini-3.8-flash	довгий	не виконано (429 Rate Limit)	—	—
--------	------------------	--------	---------------------------------	---	---

Таблиця 1.8 – Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	2.226	2.240
Всього, с	14.931	9.925

Аналіз результатів показав, що при теплому старті загальний час виконання зменшився з 14,931 с до 9,925 с. Це пояснюється тим, що модель вже була завантажена в оперативну пам'ять і не потребувала повторного завантаження перед обробкою запиту.

Крок 5. Побудувати стовпчикову діаграму (matplotlib): швидкість у символах за секунду для всіх шести комбінацій «модель × промпт». Можете запитати у LLM - як написати код для генерації стовпчикової діаграми.

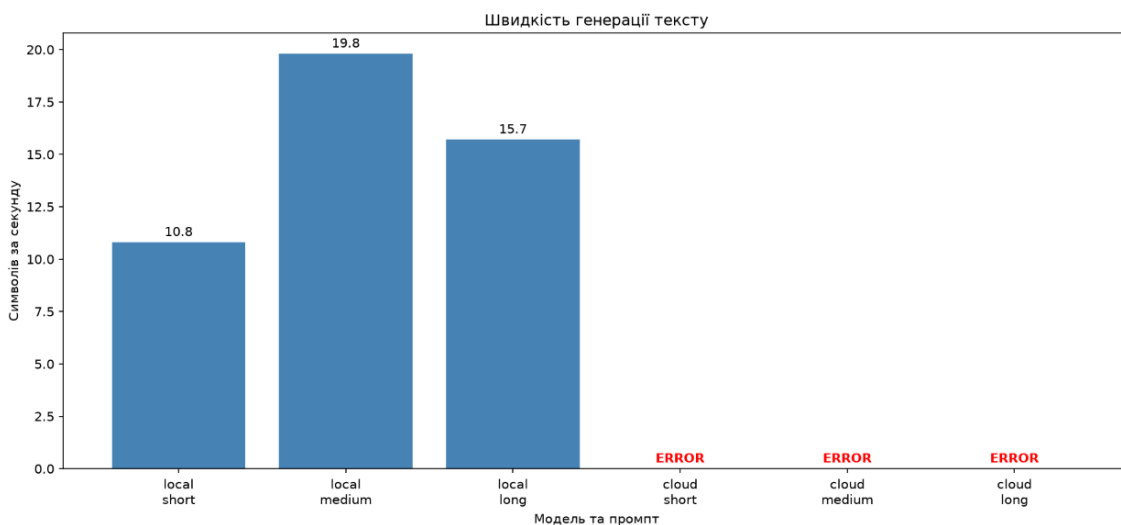


Рис. 1.11 – Швидкість генерації тексту для різних моделей та промтів

На діаграмі відображено результати вимірювання швидкості генерації тексту для локальної та хмарної моделей. Для локальної моделі успішно отримано результати для всіх трьох промтів, тоді як для хмарної моделі під час виконання експерименту було вичерпано добову квоту Gemini API, через що відповідні значення позначені як ERROR. Первинні результати хмарної моделі були успішно отримані та занесені до Таблиці 1.7, проте під час подальших повторних запусків для дослідження холодного і теплового старту файл results.csv було перезаписано, а через

перевищення ліміту API числові значення для хмарної моделі були замінені на ERROR.

Крок 6. Відповісти на питання (по 1–2 речення):

1. У якої моделі кращий TTFT і чому саме в неї?

За отриманими результатами кращий TTFT показала локальна модель. Це пояснюється відсутністю мережових затримок, оскільки обробка запиту виконується безпосередньо на локальному комп'ютері.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»?

Ні, ці показники характеризують різні аспекти роботи моделі. Модель може швидко почати відповідь, але повільніше генерувати текст, або навпаки.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промπτу до довгого?

Для короткого промπτу різниця між моделями була незначною. Для довгого промπτу порівняння виконати повністю не вдалося через обмеження безкоштовного тарифу Gemini API.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Результати окремих запусків відрізнялися через випадкові коливання навантаження системи та сервісів. Тому використання трьох повторів і медіанного значення забезпечує більш надійний результат, ніж один запуск.

5. Чому холодний старт повільніший за теплий?

Під час холодного старту модель спочатку завантажується в оперативну пам'ять, що потребує додаткового часу. При теплому старті модель уже завантажена, тому обробка запиту виконується швидше.

У звіт додати дані таблиць, діаграму та стислі відповіді на питання (2–3 речення на кожну). Стисло напишіть свої думки: який із виміряних показників важливіший 16 для інтерактивного чат-бота, а який — для фонові обробки великих обсягів тексту, і чому.

Крок 7. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиць і поставити п'ять питань:

- 1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?*

Відповідь Copilot: “Під час TTFT модель отримує запит, токенизує його, виконує перший прохід через нейронну мережу та обчислює перший токен відповіді. Цей показник вимірюють окремо, тому що він характеризує початкову затримку, яку бачить користувач до появи відповіді.”.

- 2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?*

Відповідь Copilot: “Хмарна модель потребує додаткового часу на передачу запиту через мережу, що збільшує TTFT. Водночас вона працює на потужному серверному обладнанні, тому може генерувати текст швидше після появи першого токена.”.

- 3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?*

Відповідь Copilot: “Під час холодного старту модель завантажується з диска в оперативну пам'ять і готується до роботи. Після завантаження модель залишається в пам'яті, тому наступні запуски виконуються швидше.”.

- 4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?*

Відповідь Copilot: “Під час вимірювання продуктивності використовують кілька запусків, оскільки результати можуть змінюватися через навантаження процесора, використання пам'яті, фонові процеси або мережеві затримки. Використання медіани з кількох вимірювань дозволяє зменшити вплив випадкових відхилень і отримати більш об'єктивний результат, ніж один окремий запуск.”.

- 5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?*

Відповідь Copilot: “Вхідний текст модель може обробити за один прохід. Відповідь генерується послідовно, токен за токеном, причому кожен новий токен залежить від усіх попередніх, що потребує додаткових обчислень.”.

Думки на тему який із виміряних показників важливіший для інтерактивного чат-бота, а який — для фонові обробки великих обсягів тексту, і чому.

Для інтерактивного чат-бота найважливішим показником є TTFT (Time To First Token), оскільки саме він визначає, наскільки швидко користувач побачить початок відповіді. Навіть якщо повна генерація триває довше, швидка поява перших слів створює враження оперативної роботи системи.

Для фонові обробки великих обсягів тексту важливішою є швидкість генерації (символів або токенів за секунду). У таких задачах користувач не взаємодіє з моделлю безпосередньо, тому початкова затримка менш важлива, ніж загальний час обробки великих документів або великої кількості запитів.

Крок 6. Оцінити якість роботи моделей

На даному етапі роботи оцінюється не швидкість, а якість відповідей мовних моделей. Для об’єктивного порівняння використовується фіксований набір однакових завдань і заздалегідь визначені критерії оцінювання. Оскільки мовні моделі є недетермінованими системами та можуть по-різному відповідати на однаковий запит, кожне завдання виконується двічі, а розбіжності між результатами розглядаються як додаткова характеристика якості роботи моделі.

Використанні завдання:

Кожне завдання ловить окремий тип поведінки моделі, і разом вони дають профіль її сильних і слабких місць. Нижче — що саме перевіряє кожне і на що дивитися при оцінюванні.

- **Завдання 1. Фактичні знання**

Перевіряється здатність моделі правильно відтворювати відомі факти та стабільність відповіді під час повторних запусків. Особлива увага приділяється виявленню галюцинацій, коли модель впевнено повідомляє неправдиву інформацію.

- **Завдання 2. Дотримання формату**

Перевіряється здатність моделі повертати відповідь у строго заданому форматі, придатному для автоматичної обробки програмою. Оцінюється коректність JSON та відсутність зайвого тексту або markdown-розмітки.

- Завдання 3. Робота з наданим текстом

Перевіряється вміння моделі стискати або узагальнювати наданий текст без втрати важливих даних. Основна увага приділяється збереженню всіх числових значень і фактів з оригінального тексту.

- Завдання 4. Генерація коду

Перевіряється здатність моделі генерувати програмний код, який можна безпосередньо виконати. Оцінюється правильність алгоритму, виконання всіх вимог завдання та відсутність критичних помилок.

- Завдання 5. Міркування в кілька кроків

Перевіряється здатність моделі послідовно виконувати логічні й арифметичні обчислення для отримання правильної відповіді. Важливим є не лише результат, а й коректність наведеного ходу розв'язання.

- Завдання 6. Якість української мови

Перевіряється грамотність, природність мовлення та правильність використання української термінології. Оцінюється відсутність русизмів, кальок і переходів на інші мови під час відповіді.

Як шість завдань працюють разом

Жодне окреме завдання не дає повної оцінки моделі, тому використовується набір із шести різних перевірок. У сукупності вони дозволяють визначити сильні та слабкі сторони моделі: знання фактів, дотримання формату, роботу з текстом, генерацію коду, логічні міркування та якість української мови. Саме тому важливо оцінювати не «яка модель краща», а для яких завдань кожна модель підходить найкраще. Особливо важливою характеристикою є здатність дотримуватися заданого формату відповіді, оскільки від цього залежить надійність роботи програм та агентів.

Завдання 1.5. Оцінювання якості моделей за рубрикою

Шкала оцінювання:

Таблиця 1.9 – Загальна шкала оцінювання

Бал	Критерій
-----	----------

0	Не впорався: неправильна відповідь, зламаний формат, не та мова, відмова відповідати
1	Частково: суть правильна, але є дефект — зайвий текст навколо JSON, помилка в дрібниці, кострубата мова, відсутній хід розв'язання там, де його просили
2	Виконано повністю

Таблиця 1.10 – Критерії оцінювання за завданнями

№	2 бали	1 бал	0 балів
1	Відповідь збігається з авторитетним джерелом, обидва прогони однакові	Правильна відповідь, але прогони розійшлися	Відповідь не збігається з джерелом
2	<code>json.loads()</code> парсить без правок	Парситься після видалення markdown-обгортки або вступної фрази	Не парситься взагалі
3	Два речення, усі числа збережені без спотворень	Числа правильні, але речень більше двох або додано інформацію, якої не було в тексті	Хоча б одне число спотворене або втрачене
4	Код запускається і виконує всю умову завдання	Код працює, але умову виконано частково, або навколо коду багато зайвого тексту	Не запускається або дає неправильний результат
5	Правильна відповідь і показаний хід без помилок	Правильна відповідь без ходу, або вірний підхід із дрібною помилкою в обчисленнях	Неправильна кінцева відповідь
6	Природна українська, коректна термінологія, обидва абзаци по суті	Суть правильна, але є русизми, кальки, кострубаті конструкції	Відповідь не українською або перехід на іншу мову посеред тексту

Максимум для однієї моделі — 12 балів. Для завдання 2 діє об'єктивний тест: `json.loads()` на відповіді моделі. Парситься без правок — 2 бали. Треба вирізати обгортку `json` — 1 бал. Не парситься — 0.

Таблиця 1.11 – Варіанти: завдання 1 і 2

№	Тема	Завдання 1 (факт)	Завдання 2 (формат)
1	Освіта	Рік заснування Одеського національного університету імені І. І. Мечникова	JSON: name, year, city для цього університету

2	Транспорт	Рік відкриття Київського метрополітену	JSON: city, year, lines_count для Київського метро
3	Медицина	Рік відкриття пеніциліну Флемінгом	JSON: discovery, year, scientist
4	Спорт	Рік перших сучасних Олімпійських ігор	JSON: event, year, city
5	Фінанси	Рік уведення гривні як національної валюти	JSON: currency, code, year
6	Екологія	Рік підписання Паризької кліматичної угоди	JSON: agreement, year, city
7	Кіно	Рік першого публічного кінопоказу братів Люм'єр	JSON: event, year, city
8	Туризм	Рік відкриття нинішньої будівлі Одеського оперного театру	JSON: landmark, city, year
9	Астрономія	Рік першого польоту людини в космос	JSON: event, year, person
10	Техніка	Рік випуску першого iPhone	JSON: product, year, company

Таблиця 1.12 – Варіанти: завдання 4, 5 і 6

№	Завдання 4 (код)	Завдання 5 (логіка)	Завдання 6 (українська)
1	<code>is_palindrome(s)</code> , ігнорує пробіли й регістр	25 студентів, 12 знають Python, 9 Java, 4 обидві. Скільки не знають жодної?	Чим стек відрізняється від черги
2	<code>total_weight(items)</code> — сума <code>var</code> зі списку словників	120 ящиків: 45 по 8 кг, решта по 12 кг. Вантажівка тримає 1200 кг. Скільки рейсів?	Чим контейнерне перевезення відрізняється від навалочного
3	<code>bmi(weight_kg, height_m)</code> з категорією	40 пацієнтів, 18 з температурою, 15 з кашлем, 7 з обома. У скількох немає симптомів?	Чим вірус відрізняється від бактерії
4	<code>win_rate(results)</code> зі списку “W”/“L”/“D”	8 команд, кожна з кожною по разу. Скільки матчів? При 4 матчах на день — скільки днів?	Чим спринт відрізняється від стаєрського бігу
5	<code>compound_interest(principal, rate, years)</code>	Депозит 20000 грн під 12% річних, податок 19,5% з відсотків. Скільки на руках через рік?	Чим акція відрізняється від облігації

6	sort_waste(items) за словником категорій	500 т сміття: 40% органіка, 25% пластик. Переробляють 60% пластику. Скільки тонн пластику на полігон?	Чим відновлювані джерела енергії відрізняються від невідновлюваних
7	top_rated(films, n) — n найкращих за рейтингом	Зал 180 місць, сеанс 1 год 45 хв + 15 хв перерва, робота з 10:00 до 23:00. Скільки сеансів і глядачів?	Чим документальне кіно відрізняється від художнього
8	trip_cost(days, daily_cost, transport)	Поїздка 6 днів: житло 900/добу, їжа 500/день, дорога 2400. Троє, житло й дорога порівну. Скільки заплатить кожен?	Чим хостел відрізняється від готелю
9	light_travel_time(distance_km)	Зонд 60000 км/год, відстань 480 млн км. Скільки повних діб?	Чим зоря відрізняється від планети
10	average_grade(grades) без найнижчої оцінки	Сервер обробляє 1200 запитів/хв, 15% дають помилку, з них 40% повторюються успішно. Скільки запитів втрачено за годину?	Чим оперативна пам'ять відрізняється від жорсткого диска

Текст для завдання 3: студент знаходить сам, 5–6 речень за темою свого варіанта, обов'язково з числами. Формулювання: «Стисни до двох речень, збережи всі числа.»

“Грошова реформа в Україні була проведена у вересні 1996 року. Під час реформи купуно-карбованці були замінені на гривню у співвідношенні 100000 до 1. Станом на 2025 рік Національний банк України випускає банкноти номіналом від 10 до 1000 гривень. Облікова ставка НБУ у різні роки змінювалася від 6% до понад 25%. Банківські депозити можуть приносити дохід у вигляді відсотків. Фонд гарантування вкладів фізичних осіб забезпечує захист коштів вкладників.”

Крок 1. Створити файл quality.py

```
from providers import make_client
import json
```

```
ТЕКСТ = """<текст 5-6 речень за темою вашого варіанта>"""
```

```
TASKS = {
    "1_fact": "<завдання 1 вашого варіанта>. Відповідай лише роком.",
    "2_format": "<завдання 2 вашого варіанта>. Без пояснень, без markdown, лише JSON.",
    "3_summary": "Стисни наступний текст до двох речень, збережи всі числа:
\n\n" + ТЕКСТ,
    "4_code": "<завдання 4 вашого варіанта>. Лише код.",
    "5_logic": "<завдання 5 вашого варіанта>. Покажи хід розв'язання.",
    "6_language": "<завдання 6 вашого варіанта>. Поясни українською, два абзаци.",
```

```

}

results = {}

for provider in ["local", "cloud"]:
    client, model = make_client(provider)
    results[model] = {}
    for task_id, prompt in TASKS.items():
        answers = []
        for attempt in range(2): # два прогони на завдання
            r = client.chat.completions.create(
                model=model,
                messages=[{"role": "user", "content": prompt}],
                temperature=0.7,
            )
            answers.append(r.choices[0].message.content)
        results[model][task_id] = answers
        print(f"{model} / {task_id} - готово")

with open("quality.json", "w", encoding="utf-8") as f:
    json.dump(results, f, ensure_ascii=False, indent=2)

```

Було створено файл `quality.py` для автоматичного тестування моделей за шістьма завданнями оцінювання якості. Для варіанта №5 (Фінанси) були підготовлені завдання на перевірку фактичних знань, дотримання формату, стиснення тексту, генерацію коду, логічні міркування та якості української мови. Кожне завдання виконувалося двічі для перевірки стабільності результатів.

Внесені зміни. Код із методичних вказівок було адаптовано під поточний проєкт. До `providers.py` додано підтримку моделей `llama3.2:3b`, `qwen3:4b` та `gemini-3.8-flash`, а в `quality.py` реалізовано обробку помилок (`try-except`) для коректного збереження результатів тестування у файл `quality.json`.

Фрагмент коду (try-except)

```

try:
    r = client.chat.completions.create(
        model=model,
        messages=[
            {"role": "user", "content": prompt}
        ],
        temperature=0.7,
    )

    answers.append(
        r.choices[0].message.content
    )

except Exception as e:
    answers.append(f"ERROR: {e}")
    print(f"Помилка: {model} / {task_id} / запуск {attempt + 1}")

```

Крок 2. Запустити для всіх моделей: `llama3.2:3b` , `qwen3:4b` , хмарна. Відповіді збережуться у `quality.json`.

Було виконано запуск файлу quality.py для трьох моделей. Отримані відповіді автоматично збережено у файл quality.json.

```
llama3.2:3b / 1_fact - готово
llama3.2:3b / 2_format - готово
llama3.2:3b / 3_summary - готово
llama3.2:3b / 4_code - готово
llama3.2:3b / 5_logic - готово
llama3.2:3b / 6_language - готово
qwen3:4b / 1_fact - готово
qwen3:4b / 2_format - готово
Помилка: qwen3:4b / 3_summary / запуск 2
qwen3:4b / 3_summary - готово
qwen3:4b / 4_code - готово
qwen3:4b / 5_logic - готово
Помилка: qwen3:4b / 6_language / запуск 1
Помилка: qwen3:4b / 6_language / запуск 2
qwen3:4b / 6_language - готово
gemini-3.8-flash / 1_fact - готово
Помилка: gemini-3.8-flash / 2_format / запуск 2
gemini-3.8-flash / 2_format - готово
Помилка: gemini-3.8-flash / 3_summary / запуск 1
Помилка: gemini-3.8-flash / 3_summary / запуск 2
gemini-3.8-flash / 3_summary - готово
Помилка: gemini-3.8-flash / 4_code / запуск 1
Помилка: gemini-3.8-flash / 4_code / запуск 2
gemini-3.8-flash / 4_code - готово
Помилка: gemini-3.8-flash / 5_logic / запуск 1
Помилка: gemini-3.8-flash / 5_logic / запуск 2
gemini-3.8-flash / 5_logic - готово
Помилка: gemini-3.8-flash / 6_language / запуск 1
Помилка: gemini-3.8-flash / 6_language / запуск 2
gemini-3.8-flash / 6_language - готово
quality.json створено
```

Рис. 1.12 – Процес тестування моделей у файлі quality.py

Під час тестування моделі gemini-3.8-flash виникли обмеження безкоштовного тарифу API (помилка 429 – Quota Exceeded), унаслідок чого частина завдань не була виконана.

```
"2_format": [
  "{\"currency\": \"US Dollar\", \"code\": \"USD\", \"year\": 2024}\",
  "ERROR: Error code: 429 - [{\"error\": {\"code\": 429, \"message\": \"You €
}],
"3_summary": [
  "ERROR: Error code: 429 - [{\"error\": {\"code\": 429, \"message\": \"You €
  "ERROR: Error code: 429 - [{\"error\": {\"code\": 429, \"message\": \"You €
}],
"4_code": [
  "ERROR: Error code: 429 - [{\"error\": {\"code\": 429, \"message\": \"You €
  "ERROR: Error code: 429 - [{\"error\": {\"code\": 429, \"message\": \"You €
```

Рис. 1.13 – Помилка 429 (Quota Exceeded) моделі Gemini

Крок 3. Знайти правильну відповідь на завдання 1 в авторитетному джерелі. Без цього неможливо визначити, чи галюцинує модель.

Для перевірки фактичних знань використовувалося питання:

Рік уведення гривні як національної валюти.

Правильна відповідь: **1996 рік.**

Крок 4. Перевірити завдання 2 через `json.loads()` для кожної моделі

Для оцінювання дотримання формату використовувався запит:

JSON: currency, code, year

Відповіді моделей llama3.2:3b та qwen3:4b успішно розбиралися без помилок, що підтвердило коректність JSON-формату. Для моделі gemini-3.8-flash лише один запуск повернув коректний JSON, тоді як другий завершився помилкою API 429 (Quota Exceeded), тому повна перевірка формату для цієї моделі була неможливою.

Слід зазначити, що хоча моделі llama3.2:3b та qwen3:4b повернули синтаксично правильний JSON, його зміст не відповідав темі завдання, оскільки містив дані про долар США замість гривні. Це свідчить про те, що правильний формат відповіді не гарантує правильності її змісту.

Крок 5. Відкрити `quality.json` , прочитати відповіді, виставити бали. Заповнити таблиці.

Таблиця 1.13 – Оцінка якості за рубрикою

Задання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	0	0	2
2. Формат	2	2	1
3. Стиснення	0	1	0
4. Код	1	1	0
5. Логіка	0	2	0
6. Українська	1	0	0
Разом /12	4	6	3

Примітка. Для моделі Gemini значна частина запитів завершилася помилкою 429 (Quota Exceeded), що вплинуло на підсумковий результат.

Таблиця 1.14 – Розбіжність оцінок між двома прогонами (так / ні)

Задання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	так	так	ні
2. Формат	так	ні	так
3. Стиснення	так	так	ні
4. Код	так	ні	ні
5. Логіка	так	так	ні
6. Українська	так	ні	ні

Крок 6. Відповісти на питання (по 1–2 речення):

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Найбільшу кількість балів набрала модель qwen3:4b, отримавши 6 балів із 12 можливих. Основна різниця між моделями проявилася у завданнях на логічне міркування та роботу з форматом, де qwen3:4b показала кращі результати порівняно з llama3.2:3b, а модель Gemini втратила значну кількість балів через обмеження API.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Для моделі llama3.2:3b розбіжності спостерігалися майже в усіх завданнях. Для qwen3:4b розбіжності виникли під час виконання завдань на фактичні знання, стиснення тексту та логічне міркування. Це свідчить про те, що однаковий запит може давати різні результати під час повторних запусків, а отже модель не є повністю стабільною та передбачуваною.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Так. Моделі llama3.2:3b та qwen3:4b повернули коректний JSON без markdown-розмітки, який успішно розбирався за допомогою функції `json.loads()`. Проте зміст JSON не відповідав темі завдання, оскільки замість гривні було наведено дані про долар США.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Так, проблеми спостерігалися насамперед у моделі llama3.2:3b. У відповідях зустрічалися русизми, неприродні конструкції речень, помилки у словах та змішування мов. У моделі qwen3:4b якість української мови оцінити повністю не вдалося через помилки виконання завдання "ERROR: Request timed out.", а модель Gemini не змогла завершити перевірку через обмеження безкоштовного тарифу API.

Крок 7. Обов'язковий пункт – приклад провалу. Обрати один найпоказовіший випадок і навести:

- повний текст запиту;

Депозит 20000 грн під 12% річних, податок 19,5% з відсотків. Скільки на руках через рік? Покажи хід розв'язання.

- повний текст відповіді моделі llama3.2:3b, без скорочень;

"Допустимо, ми маємо наступNING діаграму:\n\n1. Депозит 20000 грн\n2. Річна процентна ставка 12% в рік\n3. Податок 19,5% за випаси\n\nПісля року після року відбувається наступNING процес:\n\n1. Доповнення за рахунок річних процентів до депозиту.\n2. Виводження податків з випасів.\n\nПостановка problemu:\n\n1. Доповнення за рахунок річних процентів до депозиту = $12\% \times 20000 \text{ грн.} = 2400 \text{ грн.}$ \n2. Депозит після річного випасу = $20000 \text{ грн.} + 2400 \text{ грн.} = 22400 \text{ грн.}$ \n3. Податок за випаси $19,5\% \times 22400 \text{ грн.} = 4380 \text{ грн.}$ \n4. Депозит після випасу та податку = $22400 \text{ грн.} - 4380 \text{ грн.} = 18020 \text{ грн.}$ \n\nПочатковий депозит 20000 грн з підмогою річного випасу та податку на випаси amounts до 18020 грн.

- яка правильна відповідь мала бути;

Відсотки за рік:

$$20000 \times 12\% = 2400 \text{ грн}$$

Податок:

$$2400 \times 19,5\% = 468 \text{ грн}$$

Чисті відсотки:

$$2400 - 468 = 1932 \text{ грн}$$

Сума через рік:

$$20000 + 1932 = 21932 \text{ грн}$$

- пояснення, чому так сталося.

Модель правильно обчислила суму нарахованих відсотків (2400 грн), але неправильно застосувала податок. Замість оподаткування лише відсотків вона розрахувала податок від усієї суми депозиту після нарахування відсотків (22400 грн). Через цю арифметичну помилку кінцева відповідь виявилася неправильною. Цей приклад показує, що детальний хід розв'язання не гарантує правильності обчислень.

Крок 8. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиць і поставити п'ять питань:

- 1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?*

Відповідь Copilot: “На практиці використовують автоматизовані бенчмарки (MMLU, HumanEval, GSM8K та інші), експертне оцінювання відповідей людьми, тестування на реальних користувацьких сценаріях, а також А/В-тестування кількох моделей на однакових завданнях. Для агентних систем додатково перевіряють надійність виклику інструментів, стабільність формату відповідей та стійкість до помилок.”.

- 2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?*

Відповідь Copilot: “Під час навчання моделі часто бачили приклади, де JSON або програмний код оформлювалися у markdown-блоках. Через це модель вважає таке оформлення типовим і може додавати його навіть за наявності протилежної інструкції. Тому для роботи зі структурованими даними зазвичай використовують додаткову перевірку або схеми даних.”.

- 3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?*

Відповідь Copilot: “Більшість навчальних даних мовних моделей містить значно більше англійських текстів, ніж українських. Через це модель краще засвоює англійську мову, а українською частіше допускає русизми, кальки, граматичні помилки та неприродні формулювання.”.

- 4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?*

Відповідь Copilot: “Генерація тексту має випадковий характер і залежить від параметрів вибору наступних токенів. Навіть при однаковому запиті модель може обрати різні варіанти продовження тексту, що призводить до відмінностей між відповідями. Саме тому для оцінювання якості рекомендується виконувати кілька запусків одного завдання.”.

5. Що таке галюцинація мовної моделі і чому модель радше вигидає факт, ніж визнає незнання?

Відповідь Copilot: “Галюцинація — це ситуація, коли модель генерує правдоподібну, але неправильну інформацію. Модель не перевіряє факти у базі знань, а прогнозує найбільш ймовірне продовження тексту, тому за відсутності впевнених знань часто формує вигадану відповідь замість визнання власного незнання.”.

Думки наскільки надійною виявилася оцінка за рубрикою, які завдання виявились найпоказовішими для порівняння моделей, і яку модель ви обрали б для роботи з українськомовним текстом

Оцінювання за рубрикою виявилось досить надійним способом порівняння моделей, оскільки дозволяє перевіряти не лише правильність відповідей, а й стабільність результатів, дотримання формату та якості мови. Найпоказовішими виявились завдання на логічні міркування та фактичні знання, оскільки саме на них найкраще видно різницю між моделями.

Для роботи з українськомовним текстом я б обрала qwen3:4b, оскільки ця модель показала найвищий підсумковий результат серед протестованих локальних моделей та найкраще справилася із завданням на логічні міркування. Модель Gemini продемонструвала хороші результати на окремих завданнях, але повноцінне оцінювання було неможливим через обмеження безкоштовного тарифу API.

Крок 7. Дослідити вплив `temperature` і `top_p`

На результати генерації мовних моделей впливають параметри `temperature` та `top_p`, які визначають спосіб вибору наступного токена. Параметр `temperature` керує рівнем випадковості відповіді: низькі значення забезпечують більш передбачувані результати, а високі підвищують різноманітність і творчість відповідей. Параметр `top_p` обмежує набір можливих токенів для вибору, відсікаючи малоімовірні варіанти.

Для фактологічних запитів і генерації JSON зазвичай використовують низькі значення temperature, тоді як для створення ідей, назв або творчих текстів доцільно використовувати вищі значення. Метою цього етапу є дослідження впливу цих параметрів на стабільність, якість та різноманітність відповідей моделі.

Завдання 1.6. Керування випадковістю: temperature і top_p

Таблиця 1.15 – Варіанти завдань

№	Творче завдання	Фактичне завдання
1	Назва стартапу в галузі логістики	Рік заснування Національного університету «Одеська Політехніка»
2	Назва застосунку нагадувань про ліки	Рік відкриття Київського метрополітену
3	Назва платформи взаємного навчання студентів	Рік відкриття пеніциліну Флемінгом
4	Назва сервісу доставки домашньої їжі	Рік перших сучасних Олімпійських ігор
5	Назва застосунку пошуку партнерів для тренувань	Рік введення гривні
6	Назва освітнього проєкту з астрономії	Рік підписання Паризької кліматичної угоди
7	Назва застосунку обліку особистих витрат	Рік першого кінопоказу братів Люм'єр
8	Назва сервісу обміну речами	Рік першого польоту людини в космос
9	Назва рекомендаційного сервісу фільмів	Рік випуску першого iPhone
10	Назва застосунку пішохідних екскурсій	Рік відкриття будівлі Одеського оперного театру

Крок 1. Створити файл `sampling.py`

```
from providers import make_client

client, model = make_client("local")

CREATIVE = "<творче завдання вашого варіанта>"
FACTUAL = "<фактичне питання вашого варіанта>"

def run(prompt, n=5, **params):
    """Запускає промпт n разів і повертає список відповідей."""
    answers = []
    for _ in range(n):
        r = client.chat.completions.create(
            model=model,
            messages=[{"role": "user", "content": prompt}],
            **params,
```

```

    )
    answers.append(r.choices[0].message.content.strip())
return answers

```

Крок 2. Творче завдання, вплив temperature

```

for t in [0.0, 0.7, 1.5]:
    answers = run(CREATIVE, temperature=t)
    print(f"temperature={t}: унікальних {len(set(answers))} з 5")
    for a in answers:
        print(" ", a)
    print

```

Ключова метрика — кількість унікальних відповідей із п'яти. Це число йде в таблицю.

Очікуваний результат: при 0.0 одна й та сама назва п'ять разів, при 1.5 — п'ять різних, але серед них можуть трапитись безглузді.

Крок 3. Творче завдання, вплив top_p .

Той самий цикл, але температура фіксована, змінюється top_p :

```

for p in [0.1, 0.5, 1.0]:
    answers = run(CREATIVE, temperature=0.7, top_p=p)
    print(f"top_p={p}: унікальних {len(set(answers))} з 5")

```

Порівняйте з попереднім кроком: обидва параметри впливають на різноманітність, але чи однаково? Дивіться на характер відповідей, не лише на їх кількість.

Крок 4. Фактичне питання

Найважливіший експеримент. Прогоніть фактичне питання при temperature= 0.0 і temperature = 1.5 , по 5 разів:

```

for t in [0.0, 1.5]:
    answers = run(FACTUAL, temperature=t)
    print(f"temperature={

```

При високій температурі ви з великою ймовірністю побачите різні роки. Модель вигадуватиме. Це і є демонстрація того, що температура — не «регулятор розумності», а регулятор випадковості, і на фактичних задачах вона працює проти вас.

Правильну відповідь знайдіть в авторитетному джерелі, посилання вкажіть у звіті.

Крок 5. Перевірка відтворюваності.

```

answers = run(FACTUAL, n=10, temperature=0.0)
print("Усі відповіді однакові:", len(set(answers)) == 1)

```

Швидше за все збіжаться. Але гарантії немає – на це впливають особливості обчислень з плаваючою комою й паралелізація. Тримайте це в голові: у наступних лабораторних, порівнюючи дві версії агента, частина різниці може бути просто шумом, а не наслідком ваших покращень.

Крок 6. Заповнити таблиці:

Таблиця 1.16 – Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальне із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	2	5
temperature	0.7	5	—
temperature	1.5	5	0
top_p	0.1	1	—
top_p	0.5	5	—
top_p	1.0	5	—

Таблиця 1.17 – Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так/ні)	так

```
=== REPRODUCIBILITY ===
Унікальних відповідей: 1
Усі однакові: True
```

Рис. 1.14 – Результат перевірки відтворюваності моделі

Крок 7. Відповісти на питання (по 1–2 речення):

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Для агента, який повертає JSON, я б використовувала temperature=0.0 або близьке до нуля значення. Це забезпечує стабільність і зменшує ймовірність появи випадкових помилок у структурі відповіді.

2. Яке — генератору варіантів заголовків?

Для генерації заголовків або творчих ідей доцільно використовувати temperature 0.7–1.5. Висока температура підвищує різноманітність варіантів і допомагає отримати більше оригінальних ідей.

3. *Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?*

Temperature змінює розподіл ймовірностей між усіма можливими токенами, роблячи вибір більш або менш випадковим. Параметр top_p не змінює самі ймовірності, а лише обмежує набір токенів, із яких дозволено вибирати наступне слово.

4. *Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?*

Ні, теоретично температура 0 не гарантує абсолютної відтворюваності через особливості обчислень і реалізації моделі. У проведеному експерименті всі 10 відповідей збіглися, тому для даного випадку результат виявився повністю відтворюваним.

5. *Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з'явитися.*

При temperature=1.5 модель згенерувала фрагменти на кшталт:

«2. Треніруй мене - TRENIU» (просто смішно)), «можу спробуват продифолуюєлпмасіу веджерс фантузінъ.», «Звиздеміпннія для гмалбгсспа знхдфл...»

Ця відповідь втратила зміст через надто високу випадковість під час вибору токенів. При великому значенні temperature модель починає вибирати малоімовірні варіанти, що може призводити до появи вигаданих слів і беззмістовного тексту.

Крок 8. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиць і поставити п'ять питань:

1. *Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?*

Відповідь Copilot: “Параметр temperature змінює розподіл імовірностей перед вибором наступного токена. Низькі значення роблять найбільш

імовірні варіанти ще більш імовірними, а високі значення вирівнюють розподіл і підвищують шанс вибору рідкісних токенів.”.

2. *Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?*

Відповідь Copilot: “Temperature змінює співвідношення між усіма ймовірностями токенів, тоді як top_p обмежує вибір лише певною групою найбільш імовірних варіантів. Таким чином, top_p відкидає малоймовірні продовження, а temperature змінює їхню вагу.”.

3. *Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?*

Відповідь Copilot: “У багатьох випадках достатньо налаштовувати один параметр. Одночасне використання temperature і top_p дає більше контролю над генерацією, але надмірне налаштування обох параметрів може зробити поведінку моделі менш передбачуваною.”.

4. *Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?*

Відповідь Copilot: “Навіть при temperature=0 на результат можуть впливати особливості обчислень з плаваючою комою, паралельна обробка та внутрішні механізми інференсу. Тому абсолютна детермінованість не гарантується, хоча на практиці відповіді часто збігаються.”.

5. *Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?*

Відповідь Copilot: “Для витягу структурованих даних та генерації JSON зазвичай використовують temperature від 0.0 до 0.3 для максимальної стабільності. Для творчих завдань, генерації ідей або назв часто використовують значення від 0.7 до 1.5 для підвищення різноманітності відповідей.”.

Думки на тему які значення параметрів я б оберела для наступних лабораторних, де модель має повертати структуровані дані, і чому налаштування «за замовчуванням» може виявитись непридатним.

Для задач, де модель повинна повертати структуровані дані або JSON, я б використовувала temperature=0.0 та невелике значення top_p.

Проведений експеримент показав, що за високої температури модель починає генерувати випадкові та навіть беззмістовні відповіді, що може призвести до помилок під час автоматичної обробки результатів. Налаштування за замовчуванням не завжди підходять для конкретної задачі, тому параметри генерації необхідно підбирати залежно від вимог до точності та передбачуваності результату.

Крок 8. Звести все разом і зробити висновок

Заповніть підсумкову таблицю:

Таблиця 1.18 – Підсумкове порівняння моделей

Критерій	Локальна мала (3B)	Локальна більша (4B/8B)	Хмарна (Gemini)
Швидкість генерації	середня, відповідала швидше у простих задачах	низька	висока
TTFT	≈ 2.23 с	не вимірювалося окремо	≈ 2.46 с
Стабільність формату (завдання 2)	Добра, JSON парсився	Добра, JSON парсився	Частково через помилки API 429
Потрібні ресурси	Невеликі, працює на звичайному ПК	Більші вимоги до оперативної пам'яті та процесора	Локальні ресурси майже не потрібні
Куди йдуть ваші данні	Залишаються на локальному комп'ютері	Залишаються на локальному комп'ютері	Передаються на сервер провайдера
Працює без інтернету	Так	Так	Ні
Коли обирати саме її	Для простих завдань, швидких відповідей, роботи офлайн та конфіденційних даних	Для логічних задач і складніших міркувань, коли важлива приватність даних	Для максимальної якості відповідей, складних завдань, кращої української мови та коли є доступ до Інтернету

Останній рядок – головний, і його не можна заповнити одним словом. Від вас потрібне правило вибору: за яких умов ви візьмете локальну модель, попри її слабкість, а за яких — хмарну, попри залежність від мережі й передачу даних назовні.

Висновок: Мета роботи була досягнута. У процесі виконання лабораторної роботи було розгорнуто локальне середовище для роботи з великими мовними моделями на базі Ollama, досліджено принципи взаємодії з локальними та хмарними моделями через єдиний програмний інтерфейс OpenAI API, а також закріплено практичні навички роботи з мовними моделями засобами Python. Крім того, було виконано оцінювання продуктивності інференсу, проаналізовано показники швидкодії моделей та досліджено вплив параметрів temperature і top_p на стабільність та варіативність генерації тексту.

У завданні 1.1 було проведено порівняння інструктивної моделі llama3.2:3b та міркувальної моделі qwen3:4b. Результати показали, що міркувальна модель витрачає значно більше часу та токенів через виконання внутрішніх міркувань, однак обидві моделі успішно впоралися із запропонованими завданнями.

У завданні 1.2 досліджувався вплив system-повідомлення на поведінку моделей. Було встановлено, що системні інструкції суттєво впливають на мову, формат та довжину відповіді, хоча ступінь дотримання заданих обмежень залежить від конкретної моделі.

У завданні 1.3 виконано порівняння локальної та хмарної моделей через єдиний програмний інтерфейс. Отримані результати підтвердили, що використання OpenAI-сумісного API дає можливість легко змінювати провайдера моделей без зміни основної логіки програми, а хмарна модель забезпечує вищу якість відповідей.

У завданні 1.4 було виконано вимірювання продуктивності моделей за показниками TTFT та швидкості генерації тексту. Дослідження показало, що для інтерактивних чат-ботів особливо важливим є час до появи першого токена, тоді як для обробки великих обсягів даних більш значущою є загальна швидкість генерації.

У завданні 1.5 проведено оцінювання якості моделей за сукупністю критеріїв, що охоплювали фактичні знання, дотримання формату, роботу з текстом, генерацію коду, логічні міркування та якість української мови. Результати показали, що різні моделі по-різному справляються з окремими типами завдань, а якість відповіді залежить не тільки від розміру моделі, але й від специфіки поставленого запиту.

У завданні 1.6 було досліджено вплив параметрів temperature та top_p на характер генерації відповідей. Встановлено, що низькі значення temperature забезпечують високу відтворюваність і стабільність результатів, тоді як високі значення підвищують різноманітність відповідей, але можуть негативно впливати на точність фактологічної інформації.

Загалом виконана робота дозволила отримати практичний досвід роботи з локальними та хмарними мовними моделями, навчитися оцінювати їхню продуктивність і якість відповідей, а також визначати оптимальні параметри генерації для різних типів задач. Локальні моделі є доцільними для роботи з конфіденційними даними та автономного використання, тоді як хмарні моделі забезпечують вищу якість відповідей і є більш ефективними для складних завдань, що потребують глибокого аналізу та міркувань.